

CHAPTER 2

Software Quality

Concepts & Quality Models

Coverage

13 Sections

Course

Software Engineering

Deck

48 Slides

Models

ISO · McCall · FURPS · Boehm

Chapter 2 — Table of Contents

2.1	Introduction to Software Quality Concepts	p. 3	2.8	Dromey Quality Model	p.20
2.2	Dimensions of Software Quality	p. 5	2.9	ISO/IEC 9126 Quality Model	p. 2 3
2.3	Software Quality Attributes	p. 7	2.10	ISO/IEC 25010 Quality Model	p. 2 7
2.4	Quality Factors and Characteristics	p. 9	2.11	Comparison of Software Quality Models	p. 3 1
2.5	McCall's Quality Model	p. 1 1	2.12	Selection of Appropriate Quality Models	p. 3 4
2.6	Boehm's Quality Model	p. 1 4	2.13	Chapter Summary	p. 3 6
2.7	FURPS Quality Model	p. 1 7			

2.1

Introduction to Software Quality

What is quality and why does it matter in software?

Chapter 2 — Software Quality Concepts & Models

Section 2.1

What is Software Quality?

Software quality refers to the degree to which a software product meets specified requirements, satisfies user needs, and conforms to established standards. It encompasses the totality of features and characteristics that bear on its ability to satisfy stated or implied needs.

User Perspective

Software that is easy to use, reliable, meets user needs, and performs well in real-world conditions.

Developer Perspective

Software that is maintainable, testable, portable, and built using good engineering practices.

Manager Perspective

Software delivered on time, within budget, meeting specifications and conforming to standards.

 Key insight: Software quality is multi-dimensional — no single metric or attribute can capture it completely. It must be measured from multiple viewpoints.

Section 2.1

Definitions of Software Quality

IEEE Std 610.12 (1990)

The degree to which a system, component, or process meets specified requirements; meets customer or user needs or expectations.

ISO 8402 (1994)

The totality of characteristics of an entity that bear on its ability to satisfy stated and implied needs.

Crosby (1979)

"Quality is conformance to requirements." — Quality is defined by meeting the specifications, not by vague notions of goodness.

Juran (1988)

"Quality is fitness for use." — A product has quality if it successfully serves the purpose of the user during usage.

Garvin (1984)

Quality has five views: transcendent, product-based, user-based, manufacturing-based, and value-based.

2.2

Dimensions of Software Quality

The multiple facets through which quality is viewed.

Chapter 2 — Software Quality Concepts & Models

Section 2.2

Garvin's Eight Dimensions of Quality

David Garvin (1984) identified eight distinct dimensions of quality applicable to both products and software:

1. Performance



How well does the software execute its primary purpose? Speed, throughput, response time.

3. Reliability



Probability of failure-free operation for a specified period under specified conditions.

5. Durability



Amount of use before it deteriorates. For software: how long before replacement is necessary.

7. Aesthetics



Sensory qualities — look, feel, sound. Software: UI design, user experience, intuitiveness.

2. Features



Additional capabilities beyond the core function. Extra options and supplementary functions.

4. Conformance



Degree to which design matches pre-established standards, specifications, and requirements.

6. Serviceability



Speed, courtesy, competence of repair. For software: ease of bug fixing and updating.

8. Perceived Quality



Reputation and brand. Subjective assessment based on past experience and brand image.

2.3

Software Quality Attributes

Measurable characteristics of software quality.

Chapter 2 — Software Quality Concepts & Models

Section 2.3

Key Software Quality Attributes

Quality attributes (also called non-functional requirements or quality characteristics) define HOW WELL a system performs its functions:

● Functionality	Suitability, accuracy, interoperability, compliance, security
● Reliability	Maturity, fault tolerance, recoverability
● Usability	Understandability, learnability, operability, attractiveness
● Efficiency	Time behaviour (response/throughput), resource utilisation
● Maintainability	Analysability, changeability, stability, testability
● Portability	Adaptability, installability, co-existence, replaceability
● Security	Confidentiality, integrity, non-repudiation, accountability, authenticity
● Scalability	Ability to handle increasing load without degradation
● Interoperability	Ability to exchange information and cooperate with other systems

2.4

Quality Factors & Characteristics

How quality is decomposed into measurable elements.

Chapter 2 — Software Quality Concepts & Models

Section 2.4

Quality Factor Hierarchy

Software quality is structured into a three-level hierarchy: Factors → Criteria → Metrics

Quality Factors

High-level abstractions (e.g., Reliability, Maintainability)

Quality Criteria

Intermediate attributes (e.g., Fault Tolerance, Analysability)

Quality Metrics

Measurable indicators (e.g., MTBF, Defect Density, Cyclomatic Complexity)

 Quality Factors are abstract, high-level goals.
Quality Criteria define specific properties.
Quality Metrics provide objective measurements.

 Example: Factor = Reliability → Criterion = Fault Tolerance →
Metric = Mean Time Between Failures (MTBF).

2.5

McCall's Quality Model

The foundational 11-factor quality model (1977).

Chapter 2 — Software Quality Concepts & Models

Section 2.5

McCall's Quality Model — Overview

McCall, Richards, and Walters (1977) proposed one of the first comprehensive software quality models. It organises quality factors into three perspectives:

Product Revision

Ability to undergo change

- Maintainability
- Flexibility
- Testability

Product Transition

Adaptability to new environments

- Portability
- Reusability
- Interoperability

Product Operations

Operational characteristics

- Correctness
- Reliability
- Efficiency
- Integrity
- Usability

Section 2.5

McCall's 11 Quality Factors — Definitions

1	Correctness	Extent to which software meets its specifications and mission objectives
2	Reliability	Extent to which software performs its required functions without failure
3	Efficiency	Amount of computing resources required to perform a function
4	Integrity	Extent to which access to software or data by unauthorised persons can be controlled
5	Usability	Effort required to learn, operate, prepare input, and interpret output
6	Maintainability	Effort required to locate and fix a failure in operational software
7	Flexibility	Effort required to modify operational software
8	Testability	Effort required to test software to ensure it performs its functions
9	Portability	Effort required to transfer software from one environment to another
10	Reusability	Extent to which software can be used in other applications

10 **Interchangeability** — Effort required to replace software with another version

McCall's model was the first systematic approach to software quality, organised into three view points: Product Revision, Transition, and Operations

Section 2.5

McCall's Model — Strengths & Weaknesses

✓ Strengths

- First systematic quality model
- Covers multiple quality perspectives
- Clear factor-criteria-metric hierarchy
- Widely accepted and studied
- Useful for software procurement
- Provides a common vocabulary

✗ Weaknesses

- Some factors overlap (not orthogonal)
- Does not address security adequately
- Metrics not always clearly defined
- No guidance on relative importance
- Does not cover modern paradigms (OO, AI)
- Difficult to apply to large systems

2.6

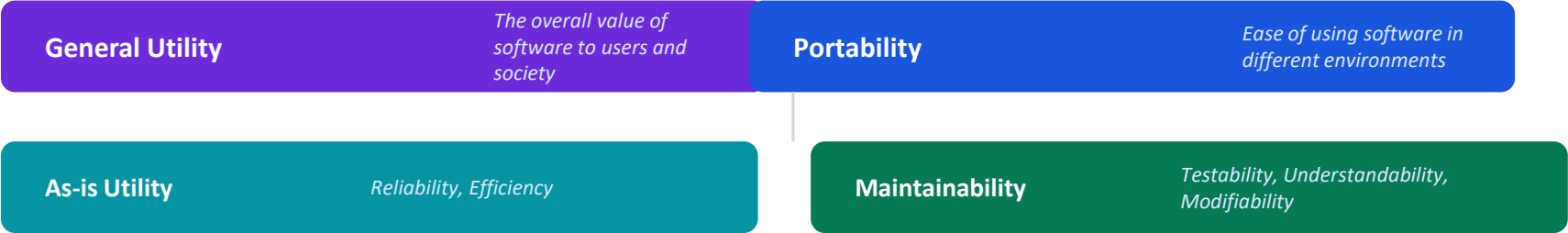
Boehm's Quality Model

A hierarchical, characteristics-based model (1978).

Section 2.6

Boehm's Quality Model — Overview

Barry Boehm (1978) extended McCall's model by providing a hierarchical structure of characteristics. It is based on utility — the overall usefulness of software.



Primitive Characteristics (15 total):

● Self-descriptiveness	● Structuredness	● Conciseness
● Legibility	● Augmentability	● Accuracy
● Completeness	● Robustness	● Consistency
● Device independence	● Accessibility	● Communicativeness
● Human engineering	● Efficiency	● Accountability

Section 2.6

Boehm's Model — Strengths & Weaknesses

✓ Strengths

- Hierarchical structure — easy to navigate
- Includes portability explicitly
- 15 primitive characteristics well-defined
- Based on utility concept — practical focus
- Supports measurement-based quality assessment
- Extended McCall with more detail

✗ Weaknesses

- No explicit measurement procedures
- Some primitive characteristics overlap
- Does not address security thoroughly
- Limited to traditional software systems
- Subjective definitions of some attributes
- Not widely adopted in practice

2.7

FURPS Quality Model

Hewlett-Packard's five-factor model (1987).

Chapter 2 — Software Quality Concepts & Models

Section 2.7

FURPS — Overview & Components

Robert Grady and Debbie Caswell (HP, 1987) developed FURPS to classify software quality attributes. Later extended to FURPS+ to include constraints.

F	Functionality <i>Features, capabilities, security, generality</i>	What the software does: required features, capabilities, security measures, and generality of functions.
U	Usability <i>Human factors, aesthetics, consistency, documentation</i>	How easy it is to use: human factors, visual aesthetics, consistency of interface, and quality of documentation.
R	Reliability <i>Frequency of failure, recoverability, predictability</i>	How dependable the software is: frequency of failure, accuracy of output, mean time to failure, ability to recover.
P	Performance <i>Speed, efficiency, throughput, response time</i>	How fast and efficiently it works: response time, throughput, accuracy, availability, resource usage.
S	Supportability <i>Testability, adaptability, maintainability, configurability</i>	How easy to maintain and support: testability, extensibility, adaptability, maintainability, configurability.



Section 2.7

FURPS+ — Extended Model & Evaluation

FURPS+ Additional Constraints:

Design (+D)

Constrains how the system must be designed. E.g., must use relational database, specific architecture.

Implementation (+I)

Constrains how the system must be built. E.g., must use Python, must comply with coding standards.

Interface (+If)

Constraints on interfaces with external systems. E.g., must connect to legacy system via specific protocol.

Physical (+Ph)

Physical constraints on hardware. E.g., must run on Raspberry Pi, maximum power consumption.

☒ Widely used in UP/RUP methodology for requirements classification

☐ Categories can overlap; no weighting mechanism provided

2.8

Dromey Quality Model

A product-property-based quality model (1995).

Section 2.8

Dromey Model — Overview & Properties

R.H. Dromey (1995) proposed a bottom-up approach linking product properties directly to quality attributes. Unlike other models, it starts from the product itself.

Module Properties

- Correctness
- Internal consistency
- No dangerous side effects
- Legal/Meaningful operators

Non-modular Properties

- Design quality
- Architectural decisions
- Component interactions
- Coupling and cohesion

Section 2.8

Dromey Model — Quality Impact & Evaluation

Dromey's model links product properties to high-level quality attributes through an impact relationship:

Product Property	Impact on Quality	Quality Attribute Affected
Well-defined module interfaces	→ Improves	Maintainability, Testability
Correct algorithms used	→ Improves	Reliability, Correctness
No global variables	→ Improves	Modularity, Maintainability
Proper error handling	→ Improves	Reliability, Robustness
Efficient data structures	→ Improves	Performance, Efficiency
Documented code	→ Improves	Maintainability, Understandability
Dangerous side-effects	→ Degrades	Reliability, Security

✓ Bottom-up approach — grounded in actual product characteristics

✗ Complex to apply; requires expertise in software analysis

2.9

ISO/IEC 9126 Quality Model

International standard quality model (1991–2001).

Chapter 2 — Software Quality Concepts & Models

Section 2.9

ISO/IEC 9126 — Overview & Six Characteristics

ISO/IEC 9126 (1991, revised 2001) defines a two-part quality model: Internal Quality & External Quality, each with six characteristics and 27 sub-characteristics.

1. Functionality

Suitability · Accuracy · Interoperability · Security · Compliance

2. Reliability

Maturity · Fault Tolerance · Recoverability · Compliance

3. Usability

Understandability · Learnability · Operability · Attractiveness · Compliance

4. Efficiency

Time Behaviour · Resource Utilisation · Compliance

5. Maintainability

Analysability · Changeability · Stability · Testability · Compliance

6. Portability

Adaptability · Installability · Co-existence · Replaceability · Compliance

Section 2.9

ISO/IEC 9126 — Quality in Use Model

ISO/IEC 9126 also defines a separate "Quality in Use" model — measuring quality from the end-user's perspective during actual use:

Effectiveness

Accuracy and completeness with which users achieve specified goals in a specified context of use.

Productivity

The relationship between the accuracy and completeness of goals achieved and the resources expended.

Safety

Acceptable levels of risk of harm to people, business, software, property, or environment.

Satisfaction

Freedom from discomfort and positive attitudes towards the use of the product.



ISO/IEC 25010 Quality Model

The SQuaRE framework — current standard (2011).

Chapter 2 — Software Quality Concepts & Models

Section 2.10

ISO/IEC 25010 — Eight Characteristics

ISO/IEC 25010 (SQuaRE, 2011) replaced ISO 9126 with an expanded 8-characteristic model and explicit Quality in Use characteristics:

1

Functional Suitability

Completeness · Correctness · Appropriateness

3

Compatibility

Co-existence · Interoperability

5

Reliability

Maturity · Availability · Fault tolerance · Recoverability

7

Maintainability

Modularity · Reusability · Analysability · Modifiability · Testability

2

Performance Efficiency

Time behaviour · Resource utilisation · Capacity

4

Usability

Recognizability · Learnability · Operability · User error protection · Aesthetics · Accessibility

6

Security

Confidentiality · Integrity · Non-repudiation · Accountability · Authenticity

8

Portability

Adaptability · Installability · Replaceability

Section 2.10

ISO/IEC 25010 — Quality in Use Characteristics

ISO/IEC 25010 defines five Quality in Use characteristics — measuring quality during actual use in a specific context:

Effectiveness*Completeness · Accuracy*

How accurately and completely users achieve their specified goals.

Efficiency*Task time · Economic productivity*

Resources expended in relation to the accuracy and completeness with which users achieve goals.

Satisfaction*Usefulness · Trust · Pleasure · Comfort*

Degree to which user needs are satisfied during use in a given context.

Freedom from Risk*Economic risk · Health/safety risk ·
Environmental risk*

Degree to which the product mitigates potential risk to financial status, human life, health, or environment.

Context Coverage*Context completeness · Flexibility*

Degree to which a product can be used effectively, efficiently, and satisfactorily in specified contexts.



2.1

1 Comparison of Quality Models

Evaluating the strengths and scope of each model.

Chapter 2 — Software Quality Concepts & Models

Section 2.11

Quality Models — Comparison Table

Model	Year	Factors/Chars	Hierarchy?	Measurable?	Scope
McCall's	1977	11 factors	Yes (3 levels)	Partially	Traditional software
Boehm's	1978	15 primitives	Yes (3 levels)	Partially	Traditional software
FURPS	1987	5 categories	Limited	Partially	OO / HP products
FURPS+	1992	5 + 4 categories	Limited	Partially	UP/RUP projects
Dromey's	1995	4 components	Yes (bottom-up)	Yes	Any software
ISO/IEC 9126	1991/ 2001	6 + sub-chars	Yes	Yes	All systems
ISO/IEC 25010	2011	8 + Quality in Use	Yes	Yes	All modern systems

✦ ISO/IEC 25010 supersedes ISO/IEC 9126 and is the current recommended standard. It addresses modern concerns like security, compatibility, and context coverage.

Section 2.11

Model Coverage Comparison — Key Attributes

Coverage of key quality attributes across the major models (✓ = explicitly covered, ○ = partially covered, X = not covered):

Attribute	McCall	Boehm	FURPS	Dromey	ISO 9126	ISO 25010
Reliability	✓	✓	✓	✓	✓	✓
Maintainability	✓	✓	✓	✓	✓	✓
Usability	✓	○	✓	○	✓	✓
Efficiency	✓	✓	✓	✓	✓	✓
Security	○	X	✓	○	✓	✓
Portability	✓	✓	X	○	✓	✓
Testability	✓	✓	✓	✓	✓	✓
Interoperability	✓	X	○	○	✓	✓
Compatibility	X	X	X	X	X	✓
Quality in Use	X	X	X	X	✓	✓

2.1

Selection of Quality Models

Choosing the right model for your context.

Chapter 2 — Software Quality Concepts & Models

Section 2.12

Criteria for Model Selection

Selecting the appropriate quality model depends on multiple contextual factors:

Project Type

Embedded, web, mobile, enterprise, safety-critical — different types have different primary quality concerns.

Stakeholder Needs

Customer/user requirements dictate which quality attributes must be prioritised.

Development Methodology

Agile teams may prefer lightweight models (FURPS); regulated industries require ISO standards.

Standards Compliance

Government contracts, medical, aviation software may require ISO/IEC 25010 compliance.

Measurement Capability

Available tools, metrics infrastructure, and team expertise limit which models are practical.

Cost & Time Constraints

Comprehensive models (ISO 25010) require significant investment. Simpler models cost less.

Model Selection Guide by Context

Recommended model based on project context:

Context / Scenario	Recommended Model	Rationale
Academic/research projects	McCall's or Boehm's	Well-documented, widely studied, good learning models
Object-oriented software (UML/UP)	FURPS+ model	Designed for OO; integrates with RUP methodology
Web / mobile / SaaS applications	ISO/IEC 25010	Covers modern concerns: security, compatibility, efficiency
Safety-critical systems (medical/aviation)	ISO/IEC 25010	International standard; required by regulators
Legacy system maintenance	McCall's + custom	Focus on maintainability, testability, portability factors
Procurement / vendor evaluation	ISO/IEC 25010	Objective, internationally recognised criteria
Agile / startup projects	FURPS (simplified)	Lightweight; quick to apply; fits sprint-based work
Research into product properties	Dromey's model	Bottom-up, links properties directly to quality

Chapter Summary

Key takeaways from Chapter 2.

Chapter 2 — Software Quality Concepts & Models

Section 2.13

Key Concepts — Chapter 2 Summary

● Software Quality:	Multi-dimensional — no single definition. Viewed from user, developer, and manager perspectives. Based on conformance to requirements and fitness for use.
● Garvin's Dimensions:	Eight dimensions: Performance, Features, Reliability, Conformance, Durability, Serviceability, Aesthetics, Perceived Quality.
● Quality Attributes:	Non-functional requirements: Functionality, Reliability, Usability, Efficiency, Maintainability, Portability, Security, Scalability.
● Quality Hierarchy:	Quality Factors → Quality Criteria → Quality Metrics. Three levels from abstract to measurable.
● McCall's (1977):	11 factors across 3 views: Product Operations, Revision, Transition. First systematic model.
● Boehm's (1978):	15 primitive characteristics. Hierarchical. Based on utility concept. Includes portability explicitly.
● FURPS (1987):	Five categories: Functionality, Usability, Reliability, Performance, Supportability. FURPS+ adds constraints.
● Dromey's (1995):	Bottom-up approach. Links product properties to quality attributes. Four components.
● ISO/IEC 9126:	Six characteristics. Two-part: Internal/External Quality + Quality in Use. International standard (1991–2001).
● ISO/IEC 25010:	Current standard (2011). Eight product quality characteristics + five Quality in Use characteristics. SQuaRE framework.

Key Terms & Definitions

Software Quality	The degree to which software meets requirements, satisfies needs, and conforms to standards.
Quality Factor	A high-level abstraction representing a major quality concern (e.g., Reliability, Maintainability).
Quality Criterion	An intermediate attribute that contributes to a quality factor (e.g., Fault Tolerance, Testability).
Quality Metric	A measurable indicator of a quality criterion (e.g., MTBF, Defect Density, Cyclomatic Complexity).
SQuaRE	System and Software Quality Requirements and Evaluation — ISO/IEC framework replacing ISO 9126.
Quality in Use	Quality experienced by end users when using software in a specific real-world context.
FURPS	Functionality, Usability, Reliability, Performance, Supportability — HP quality classification.
Quality Attribute	A non-functional characteristic describing how well the software performs its functions.
ISO/IEC 25010	Current international standard (2011) defining 8 product quality + 5 quality-in-use characteristics.
Quality Model	A framework that categorises and defines software quality characteristics and their relationships.

Review & Practice Questions

Q1. Define software quality. How does the definition differ between users, developers, and managers?

Q2. List and explain Garvin's eight dimensions of software quality with examples.

Q3. Distinguish between Quality Factors, Quality Criteria, and Quality Metrics with examples.

Q4. Describe McCall's quality model. List all 11 factors and classify them into the three product views.

Q5. Compare McCall's model with Boehm's model. What are the key differences?

Q6. Explain the FURPS quality model. What additional categories does FURPS+ include?

Q7. What is Dromey's quality model? How does its bottom-up approach differ from other models?

Q8. Describe the six quality characteristics of ISO/IEC 9126 and their sub-characteristics.

Q9. What improvements does ISO/IEC 25010 introduce over ISO/IEC 9126? List the eight characteristics.

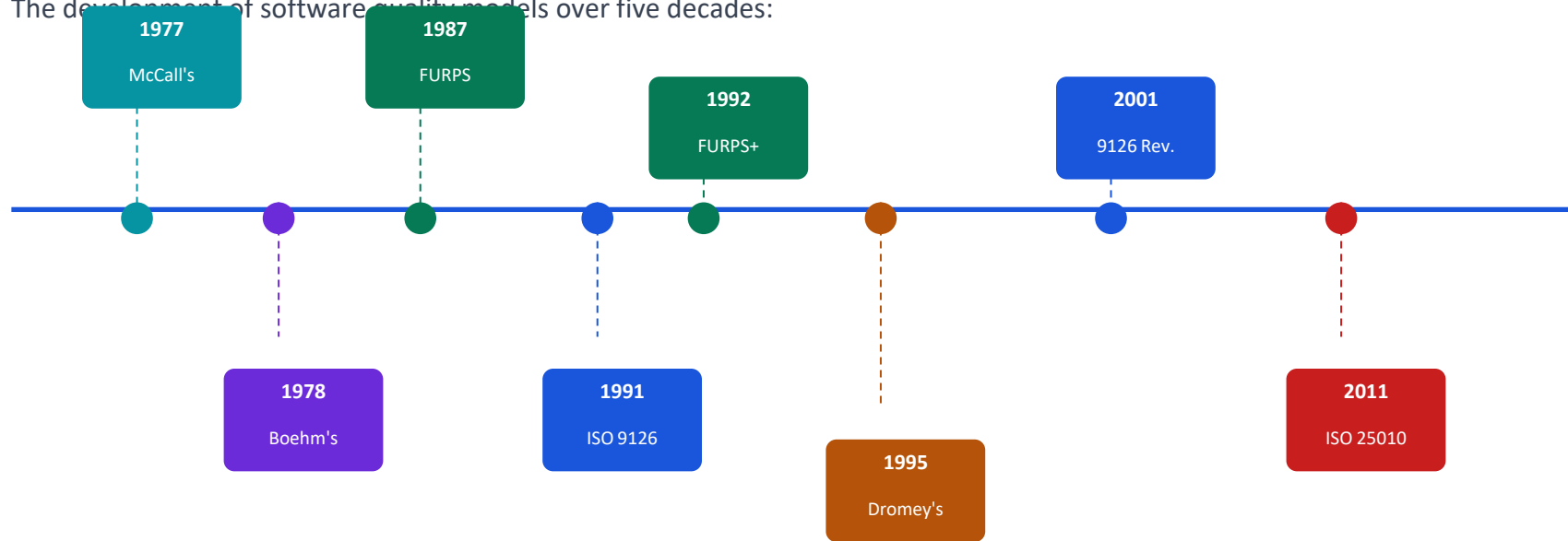
Q10. Compare any three software quality models in terms of scope, hierarchy, and measurability.

Q11. Under what circumstances would you choose ISO/IEC 25010 over FURPS for a project?

Q12. What is "Quality in Use"? How is it defined in ISO/IEC 9126 vs. ISO/IEC 25010?

Evolution of Software Quality Models

The development of software quality models over five decades:



🔗 Key evolution: McCall (1977) → ISO 9126 (1991) → ISO 25010 (2011). Each generation addressed gaps in the previous model, adding more characteristics, better measurability, and modern concerns like security and Quality in Use.

Quality Models — Quick Reference Cards

McCall's

1977

- 11 factors, 3 perspectives
- Product Operations: Correctness, Reliability, Efficiency, Integrity, Usability
- Product Revision: Maintainability, Flexibility, Testability
- Product Transition: Portability, Reusability, Interoperability

Boehm's

1978

- 2 high-level: General Utility + Portability
- 3 mid-level: As-is Utility + Portability + Maintainability
- 15 primitive characteristics
- Bottom-up from testability and understandability

FURPS+

1987/92

- F — Functionality
- U — Usability
- R — Reliability
- P — Performance
- S — Supportability
- + Design, Implementation, Interface, Physical constraints

Dromey's

1995

- Product property → Quality attribute links
- Module vs. non-modular properties
- Bottom-up approach from product
- 4 component types analysed

ISO/IEC 9126

2001

- 6 characteristics, 27 sub-characteristics
- Functionality · Reliability · Usability
- Efficiency · Maintainability · Portability
- + 4 Quality in Use: Effectiveness, Productivity, Safety, Satisfaction

ISO/IEC 25010

2011

- 8 product quality characteristics
- Functional Suitability · Performance · Compatibility
- Usability · Reliability · Security · Maintainability · Portability
- + 5 Quality in Use characteristics

Exam Tips — Chapter 2

Focus areas for examinations:



Definitions

- Define software quality from IEEE, ISO, Crosby, and Juran perspectives
- Explain the quality factor–criteria–metric hierarchy
- Define Quality in Use vs. Internal/External Quality



McCall's Model

- Name all 11 factors — memorise the three views
- Classify factors: which view does each belong to?
- Know the strengths and weaknesses of the model



ISO Standards

- Differences between ISO 9126 and ISO 25010
- What new characteristics does 25010 add over 9126?
- Name all 8 product quality characteristics of ISO 25010



Comparison

- Compare any two models: structure, scope, measurability
- When would you use FURPS vs ISO 25010?
- Which model covers security explicitly and comprehensively?

Chapter 2 — Concept Relationships

References & Further Reading

- **McCall, J.A., Richards, P.K., & Walters, G.F. (1977)**
Factors in Software Quality, RADC-TR-77-369, Rome Air Development Center.
- **Boehm, B.W., Brown, J.R., & Lipow, M. (1978)**
Characteristics of Software Quality, North-Holland, Amsterdam.
- **Grady, R.B. & Caswell, D.L. (1987)**
Software Metrics: Establishing a Company-Wide Program, Prentice-Hall.
- **Dromey, R.H. (1995)**
"A Model for Software Product Quality," IEEE Transactions on Software Engineering, 21(2), pp. 146–162.
- **ISO/IEC 9126 (2001)**
Software Engineering — Product Quality, International Organization for Standardization.
- **ISO/IEC 25010 (2011)**
Systems and Software Engineering — SQuaRE — System and Software Quality Models, ISO/IEC.
- **Garvin, D.A. (1984)**
"What Does Product Quality Really Mean?" Sloan Management Review, 26(1), pp. 25–43.
- **Pressman, R.S. & Maxim, B.R. (2019)**
Software Engineering: A Practitioner's Approach (9th ed.), McGraw-Hill.

McCall's Model vs. ISO/IEC 25010 — Detailed Comparison

Aspect	McCall's (1977)	ISO/IEC 25010 (2011)
Year published	1977	2011
Number of top-level factors	11 quality factors	8 quality characteristics
Hierarchy	Factors → Criteria → Metrics	Characteristics → Sub-characteristics
Quality in Use	Not included	5 characteristics explicitly defined
Security treatment	Integrity (limited)	Full security characteristic: 5 sub-chars
Compatibility	Interoperability only	Compatibility characteristic: 2 sub-chars
Measurability	Partially defined	Well-defined with measurement framework
International standard	No	Yes — ISO/IEC 25010
Coverage of modern systems	Limited	Designed for modern, complex systems
Applicability	Traditional software	All types: web, mobile, embedded, AI
Complexity	Relatively simple	More comprehensive and complex
Regulatory compliance	Not recognised	Accepted by regulatory bodies worldwide

ISO/IEC 9126 vs. ISO/IEC 25010 — What Changed?

ISO/IEC 25010 (2011) replaced ISO/IEC 9126 (2001) with significant enhancements:

ISO/IEC 9126 (2001)	Change	ISO/IEC 25010 (2011)
6 product quality characteristics	Expanded to →	8 product quality characteristics
Functionality	Renamed & restructured →	Functional Suitability
Efficiency	Renamed →	Performance Efficiency
No Compatibility characteristic	New addition →	Compatibility (Co-existence, Interoperability)
Security = sub-char of Functionality	Promoted to top level →	Security (5 sub-characteristics)
4 Quality in Use chars	Expanded to →	5 Quality in Use characteristics
Effectiveness, Productivity, Safety, Satisfaction	Added new →	+ Context Coverage characteristic
No Usability sub-chars for accessibility	Added →	Accessibility as explicit sub-characteristic
Reliability sub-chars	Added →	Availability as explicit sub-characteristic